SoftwareSerial()

To create a new instance of this library we need to use the `SoftwareSerial()` function to give it a name and specify the RX and TX pin numbers. The syntax follows:

```
SoftwareSerial name(rxPin, txPin)
```

The first thing we need to set is the name for our instance. We could use `mySerial`, `softSerial`, or anything else we want that's not already taken, including `George`, but it's probably a good idea to give it a name that relates to the device connected to the serial port and to avoid any confusion should we want to use the hardware Serial functions as well. In our example we used the following statement:

```
SoftwareSerial rfid(2,3);
```

Here, we are calling our software serial port `rfid` to remind us which device that we're talking to. While it is possible to have multiple instances of the SoftwareSerial library, we can only read from or send to one at a time. The timing of these operations could get tricky as we get more devices going at the same time. For example, if we were sending data through a wireless adapter on one serial port, we might miss an incoming tag that needed to be read from the RFID serial port.

The final two parameters that we specified in our example as 2 and 3 are the pin numbers that we will use as receive, RX, and transmit, TX, pins. These can be any of the I/O pins on the Arduino Uno, so long as they are not in use by anything else.

begin()

The `begin()` function is identical in form to the same function found in the Serial library. Give the function an instance name, in our example `rfid`, and specify a data rate in bits per second and all should be fine. Although because we are using software on an already burdened microcontroller to emulate what is often done with dedicated hardware, things don't always go according to plan. Data rates of 300 bps or 1200 bps are nearly unusable and you might be pushing your luck with 57,600 bps and over. Just keep in mind that when specifying data rates for software serial ports, if unusual behavior crops up, this data rate may be suspect.

flush()

While the `flush()` function is the same for the Serial library, we didn't talk about it before because we didn't have a good reason for it. The syntax is another simple one, as follows:

```
name.flush()
```

Where `name` is the name of your serial instance and beyond that there are no parameters and nothing is returned. This function's sole purpose in life is to clear the serial buffer of any waiting data or in the case of SoftwareSerial, to clear any forthcoming data on the serial line. We used this function in our most recent sketch so that we didn't need to worry about the last three characters that the RFID reader sends down the line, whether we want it to or not. We have to do something to either read this data and put it somewhere or simply get rid of it because our code is executed so quickly that these remaining bytes will be taken for tag information and consequently confuse things. Since we know there are 12 bytes that we are interested in and our communications are reasonably accurate, this gives us a way to just get rid of the extra data.